

CIS-11 Project Documentation

Test Score Calculator

Team Name: The Assemblers

Team Members: J Miguel Belarmino
Joshua Barajas
Keisha Coleen (Cole) Wagas

Project: Option B Test Score Calculator

Date: May 22, 2026

Advisor: Kasey Nguyen, PhD

Part I — Application Overview

1. Objectives

The objective of this project is to develop a program that accepts five two-digit test scores from a user and automatically computes the minimum score, maximum score, average score, and corresponding letter grade. The program eliminates the need for manual calculation, reducing the likelihood of arithmetic errors and saving time for students and instructors who need a quick summary of test performance.

The program achieves this by prompting the user for input one score at a time, storing the values, and performing all calculations internally before displaying a clean, labeled summary of results. If this program were not developed, users would be required to calculate these statistics by hand or rely on a separate tool.

The people who benefit most from this program are students and instructors who need a fast, straightforward tool for evaluating test score data without external software.

2. Business Process

The program operates as a single-session, interactive command-line application running in the LC-3 simulator environment. The process is as follows:

1. The program launches and displays a welcome message: `TEST SCORE CALCULATOR`.
2. The user is prompted to enter five test scores, one at a time. Each score must be a two-digit integer between 00 and 99.
3. After all five scores are entered, the program processes the data internally without any further user interaction.
4. The program displays the minimum score, maximum score, average score, and letter grade, each on a labeled line.
5. The program halts.

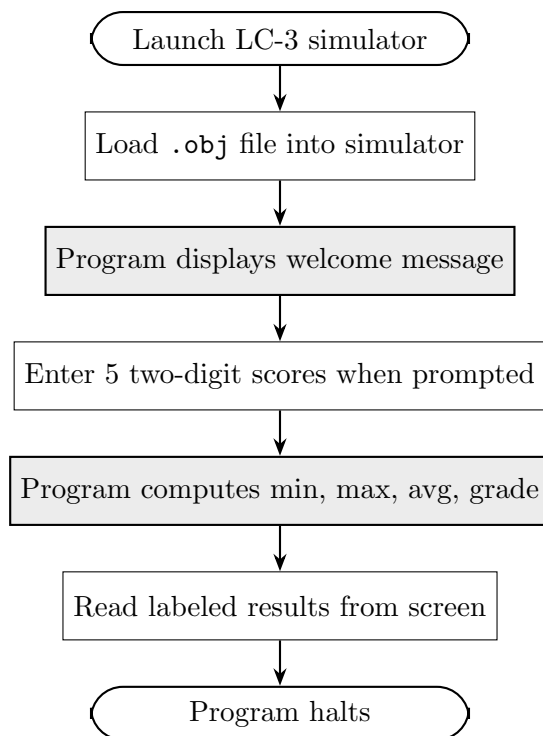
No existing system is being replaced. This is a standalone tool introduced to demonstrate assembly-level programming. The business value lies in its ability to produce accurate results instantly following the final input.

3. User Roles and Responsibilities

There is one user role for this application, the End User. This role encompasses any student or instructor who runs the program in the LC-3 simulator.

Responsibilities of the End User:

- Launch the program in the LC-3 simulator environment.
- Enter five valid two-digit test scores when prompted.
- Read and interpret the output results displayed by the program.



White boxes = End User actions Gray boxes = System processes

4. Production Rollout Considerations

This program is intended to run within the LC-3 simulator, a software environment used in computer science and engineering coursework. No installation beyond the simulator itself is required.

- **Environment:** LC-3 simulator, loaded via a `.obj` file assembled from the LC-3 source code.
- **Data population:** No pre-existing data is needed. All input is provided live by the user at runtime.
- **Transaction volume:** The program handles exactly five input values per session. Each session is independent with no persistent state between runs.
- **Rollout strategy:** The assembled `.obj` file is distributed to users who load it directly into the LC-3 simulator. No network access, installation, or configuration is required.

5. Terminology

LC-3 Little Computer 3: a simplified educational computer architecture used to teach low-level programming concepts. This program is written in LC-3 assembly language and executed in an LC-3 simulator.

ASCII American Standard Code for Information Interchange: a character encoding standard that maps letters, digits, and symbols to numeric values. This program uses

ASCII to read digit characters from the keyboard and convert them to numeric values for computation.

MIN The lowest score among the five values entered by the user. Computed by the **FINDMIN** subroutine.

MAX The highest score among the five values entered by the user. Computed by the **FINDMAX** subroutine.

AVG The arithmetic average of all five scores, calculated by dividing their sum by 5 using repeated subtraction. Computed by the **FINDAVG** subroutine.

GRADE The letter grade corresponding to the computed average, assigned according to the scale: 90–100 = A, 80–89 = B, 70–79 = C, 60–69 = D, 0–59 = F. Determined by the **FINDGRADE** subroutine.

Two-digit score A numeric score in the range 00–99, entered as two consecutive digit characters from the keyboard.

Subroutine A reusable block of LC-3 instructions called from the main program and returning control upon completion. Each major step of this program is implemented as a subroutine.

Part II — Functional Requirements

1. Statement of Functionality

The application will include all of the following functions and no others:

- **Score Input (INPUTS):** The program will prompt the user to enter exactly five two-digit test scores. Each score is read as two individual digit characters from the keyboard, converted from ASCII to numeric values, and stored in a five-element array. A running total (**SUM**) is accumulated during input.
- **Minimum Calculation (FINDMIN):** The program will scan all five stored scores and identify the lowest value, storing it in the variable **MIN**.
- **Maximum Calculation (FINDMAX):** The program will scan all five stored scores and identify the highest value, storing it in the variable **MAX**.
- **Average Calculation (FINDAVG):** The program will compute the integer average of the five scores by dividing **SUM** by 5 using repeated subtraction, storing the result in **AVG**.
- **Grade Assignment (FINDGRADE):** The program will compare **AVG** against defined grade thresholds and store the corresponding ASCII letter character in **GRADE**. The scale is: 90–100 = A, 80–89 = B, 70–79 = C, 60–69 = D, 0–59 = F.
- **Results Output (PRINTRESULTS):** The program will display the minimum, maximum, and average scores as readable two-digit numbers, followed by the letter grade, each on a labeled line.

The application will **not** include the following:

- Support for scores outside the range 00–99 or scores with more than two digits.
- File input, output, or any form of persistent data storage between sessions.
- Support for more or fewer than five scores per session.
- A graphical user interface of any kind.

2. Scope

This project consists of a single release containing all six functional steps. The current implementation status is as follows:

- **Step 1 — INPUTS:** Implemented.
- **Step 2 — FINDMIN:** Implemented.
- **Step 3 — FINDMAX:** Not yet implemented.
- **Step 4 — FINDAVG:** Not yet implemented.
- **Step 5 — FINDGRADE:** Not yet implemented.
- **Step 6 — PRINTRESULTS:** Not yet implemented.

All six steps are planned for delivery in the final submission of this project.

3. Performance

The program runs entirely within the LC-3 simulator on a standard personal computer. Given the minimal computational load (five scores, fixed arithmetic operations), the following performance characteristics apply:

- All computations (minimum, maximum, average, grade) will complete within one second of the fifth score being entered, under normal simulator conditions.
- The program will process exactly five scores per session with no degradation in response time regardless of the values entered.

4. Usability

The program is designed for use by individuals familiar with running programs in the LC-3 simulator. The following usability goals apply:

- Each score entry is preceded by a clear prompt (**Enter score:**) so the user always knows when input is expected.
- Scores are entered one per line for clarity.
- All output values are labeled (e.g., **Minimum:** , **Maximum:** , **Average:** , **Grade:**) so results require no interpretation.

- The program requires no prior configuration or setup beyond loading it into the LC-3 simulator.

5. Requests for Enhancements

Date	Enhancement	Requested By	Notes	Priority	Status

Part III — Appendices

Appendix A: Pseudocode

The pseudocode below documents all six subroutines of the Test Score Calculator. It demonstrates the required structural elements: **subroutines/functions** (each STEP is a called subroutine), **iteration** (loops in INPUTS, FINDMIN, FINDMAX, and FINDAVG), and **branching** (the conditional chain in FINDGRADE).

```
MAIN
  Program starts at x3000
  Print welcome message ("TEST SCORE CALCULATOR")
  Call INPUTS           ; Step 1 - implemented
  Call FINDMIN           ; Step 2 - implemented
  Call FINDMAX           ; Step 3 - not yet implemented
  Call FINDAVG           ; Step 4 - not yet implemented
  Call FINDGRADE         ; Step 5 - not yet implemented
  Call PRINTRESULTS      ; Step 6 - not yet implemented
  HALT

INPUTS
  Save registers R1 through R7
  R1 = pointer to start of SCORES array
  R2 = 5 (loop counter)
  Loop while R2 > 0:
    Print "Enter score: "
    Read first digit character, echo to screen
```

```
    Subtract ASCII '0' (x0030) to convert to number, store in R4
    Multiply R4 by 10 via repeated addition to get tens digit value
    Read second digit character, echo to screen
    Subtract ASCII '0' to convert to number, store in R6
    Final score = (R4 x 10) + R6
    Store final score in SCORES array at current position
    Advance array pointer by 1
    Add final score to SUM
    Print newline
    Decrement R2 by 1
End Loop
Restore registers R1 through R7
Return to Main
```

FINDMIN

```
    Save registers R1 through R7
    R1 = pointer to start of SCORES array
    Load SCORES[0] into R3
    Save R3 into MIN (assume first score is minimum)
    Advance pointer to SCORES[1]
    R2 = 4 (check remaining 4 scores)
    Loop while R2 > 0:
        Load current score into R4
        Load current MIN into R3
        Compute R4 - R3
        If result is negative (R4 < MIN):
            Save R4 into MIN
        Advance array pointer by 1
        Decrement R2 by 1
    End Loop
    Restore registers R1 through R7
    Return to Main
```

FINDMAX

```
    Save registers R1 through R7
    R1 = pointer to start of SCORES array
    Load SCORES[0] into R3
    Save R3 into MAX (assume first score is maximum)
    Advance pointer to SCORES[1]
    R2 = 4 (check remaining 4 scores)
    Loop while R2 > 0:
        Load current score into R4
        Load current MAX into R3
        Compute R4 - R3
        If result is positive (R4 > MAX):
            Save R4 into MAX
        Advance array pointer by 1
```

```
    Decrement R2 by 1
End Loop
Restore registers R1 through R7
Return to Main
```

FINDAVG

```
Save registers R1 through R7
Load SUM into R1
R2 = 0    (will hold the average)
R3 = -5   (subtraction constant)
Loop:
    Add R3 to R1  (R1 = R1 - 5)
    If R1 < 0: exit loop
    Increment R2 by 1
End Loop
Save R2 into AVG
Restore registers R1 through R7
Return to Main
```

FINDGRADE

```
Save registers R1 through R7
Load AVG into R1
If AVG >= 90: GRADE = 'A'  (store ASCII x0041)
Else if AVG >= 80: GRADE = 'B'  (store ASCII x0042)
Else if AVG >= 70: GRADE = 'C'  (store ASCII x0043)
Else if AVG >= 60: GRADE = 'D'  (store ASCII x0044)
Else: GRADE = 'F'  (store ASCII x0046)
Restore registers R1 through R7
Return to Main
```

PRINTRESULTS

```
Save registers R1 through R7
Print "Minimum: "
Load MIN, extract tens and ones digits via repeated subtraction
Convert each digit to ASCII by adding x0030, print each digit
Print newline
Print "Maximum: "
Load MAX, repeat same digit extraction and conversion
Print newline
Print "Average: "
Load AVG, repeat same digit extraction and conversion
Print newline
Print "Grade: "
Load GRADE (already stored as ASCII character), print it
Print newline
Restore registers R1 through R7
Return to Main
```



```
DATA SECTION
WELCOME    .STRINGZ  "TEST SCORE CALCULATOR\n"
PROMPT     .STRINGZ  "Enter score: "
NEWLINE    .STRINGZ  "\n"
LBLMIN     .STRINGZ  "Minimum: "
LBLMAX     .STRINGZ  "Maximum: "
LBLAVG     .STRINGZ  "Average: "
LBLGRD     .STRINGZ  "Grade: "
ASCIIZERO  .FILL  x0030
SCORES     .BLKW  5
SUM        .FILL  #0
MIN        .FILL  #0
MAX        .FILL  #0
AVG        .FILL  #0
GRADE      .FILL  #0
.END
```

Listing 1: Test Score Calculator Full Pseudocode